PYSPARK
ZERO TO CERTIFICATION

Consolidated training manual | Days 01-20 of 28

Databricks Certified Associate Developer for Apache Spark (PySpark)
Theory | runnable examples | guided labs | solutions | exam-style checks | common traps

Edition: November 2026. Chapters 1-18 are expanded reconstructions of the previously delivered curriculum, not verbatim transcripts. Chapters 19-20 follow the Spark Connect lessons. Code samples are educational and may require a configured Spark runtime, version-compatible packages or persistent storage.

CONTENTS

Day 01 - Spark Foundations and Distributed Computing

Day 02 - SparkSession, Schemas and DataFrames

Day 03 - DataFrame Transformations and Null Semantics

Day 04 - Catalyst, Lazy Evaluation and Physical Execution

Day 05 - Joins, Join Types and Broadcast Strategies

Day 06 - Spark SQL and Temporary Views

Day 07 - Reading, Writing and File Formats

Day 08 - Analytical Window Functions

Day 09 - Performance Tuning and Shuffles

Day 10 - Advanced Transformations and Complex Columns

Day 11 - Nested Data and Higher-Order Functions

Day 12 - Python UDFs, Pandas UDFs and Serialization

Day 13 - Advanced Pandas UDFs and Grouped Operations

Day 14 - Intermediate Review and Assessment

Day 15 - Structured Streaming Fundamentals

Day 16 - Streaming Sources, Sinks, Triggers and Output Modes

Day 17 - Event-Time Windows, Watermarks and Stateful Processing

Day 18 - Streaming Checkpoints, Recovery and foreachBatch

Day 19 - Spark Connect Architecture

Day 20 - Spark Connect Applications, Testing and Remote Data

How to use this manual

Each chapter is designed for a 2-4 hour study block: read the conceptual explanation, run the example, complete the lab without looking at the solution, then use the exam check and traps to test understanding. Code samples generally assume a SparkSession named spark. Days 19-20 require a separate compatible Spark Connect server.

DAY 01 | Spark Foundations and Distributed Computing

Learning focus

Spark distributes work across partitions, allowing executors to process data in parallel. The driver creates the application plan and coordinates tasks. Spark is suited to large-scale transformations, not small single-row transactional workloads.

Core concepts and mental model

Driver: orchestrates jobs and tasks. Executor: executes tasks and holds cached partitions. Partition: unit of parallel data processing. Transformation: lazy operation. Action: requests computation.

Runnable PySpark example

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("day01").getOrCreate()

df = spark.range(0, 100)

print(df.count())

print(df.rdd.getNumPartitions())  # traditional PySpark only
```

Guided practical lab

Create a range of 1000 integers; count even IDs.

Worked solution / verification

```
even = spark.range(1000).filter("id % 2 = 0")

assert even.count() == 500
```

Certification-style knowledge check

Q1: Which component schedules tasks? A) Driver B) Executor C) CSV. Answer: A. Q2: Does filter execute immediately? Answer: No; it is lazy.

Common exam trap

Do not confuse an executor with a partition; multiple partitions may run on one executor.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 02 | SparkSession, Schemas and DataFrames

Learning focus

A SparkSession is the main entry point for DataFrame and SQL APIs. DataFrames are distributed, schema-aware collections of rows. Explicit schemas reduce inference errors and improve reproducibility.

Core concepts and mental model

StructType and StructField define column types and nullability. show() displays samples; printSchema() inspects types; createDataFrame() constructs data.

Runnable PySpark example

```
from pyspark.sql.types import StructType,StructField,StringType,DoubleType

schema=StructType([StructField("desk",StringType(),False),StructField("pnl",DoubleType(),True)])

df=spark.createDataFrame([("Equity",10.0),("Rates",None)],schema)

df.printSchema();df.show()
```

Guided practical lab

Build a trade DataFrame with trade_id, desk, and pnl; inspect schema and nulls.

Worked solution / verification

```
from pyspark.sql import functions as F

assert df.filter(F.col("pnl").isNull()).count() == 1
```

Certification-style knowledge check

Q1: Which object defines a DataFrame schema? Answer: StructType. Q2: Does count() trigger execution? Answer: Yes.

Common exam trap

InferSchema is convenient but can infer undesirable types; use explicit schemas in production.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 03 | DataFrame Transformations and Null Semantics

Learning focus

select, filter, withColumn, drop, and distinct transform DataFrames without mutating the original. Spark SQL null follows three-valued logic; comparisons with null do not evaluate to true.

Core concepts and mental model

Use col(), lit(), when(), coalesce(), isNull(), and isNotNull(). filter() selects rows; select() selects expressions.

Runnable PySpark example

```
from pyspark.sql import functions as F

df=spark.createDataFrame([("Equity",100.0),("Rates",None)], ["desk","pnl"])

clean=df.withColumn("pnl",F.coalesce("pnl",F.lit(0.0)))

clean=clean.withColumn("sign",F.when(F.col("pnl")>0,"profit").otherwise("other"))

clean.show()
```

Guided practical lab

Create a flag for negative P&L and fill missing P&L with zero; explain whether filling nulls is valid for your business.

Worked solution / verification

```
result=df.withColumn("loss",F.col("pnl")<0)

assert result.count()==2
```

Certification-style knowledge check

Q1: Is df.filter(col("pnl") == None) correct? Answer: No; use isNull(). Q2: Does withColumn mutate df? Answer: No.

Common exam trap

Null is not the same as zero; do not silently substitute zero in financial data.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 04 | Catalyst, Lazy Evaluation and Physical Execution

Learning focus

Spark builds a logical plan for DataFrame transformations, analyzes and optimizes it with Catalyst, then selects a physical plan. Actions initiate jobs; wide transformations may introduce shuffle boundaries.

Core concepts and mental model

Logical plan describes what to calculate. Physical plan describes how. explain() shows plans. Narrow transformations often avoid a shuffle; groupBy usually shuffles.

Runnable PySpark example

```
from pyspark.sql import functions as F

df=spark.range(100000)

q=df.filter(F.col("id")%2==0).groupBy((F.col("id")%10).alias("bucket")).count()

q.explain("formatted")

q.show()
```

Guided practical lab

Use explain() to identify an Exchange operator; compare filter-only with groupBy queries.

Worked solution / verification

```
spark.range(100).filter("id < 10").explain("formatted")
```

Certification-style knowledge check

Q1: What optimizes SQL/DataFrame plans? Answer: Catalyst. Q2: Which operation often shuffles? Answer: groupBy.

Common exam trap

Do not assume every transformation creates a job or that all wide operations use identical shuffle strategies.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 05 | Joins, Join Types and Broadcast Strategies

Learning focus

Joins combine DataFrames by matching keys. Inner, left, right, full, semi and anti joins have different row-preservation semantics. Broadcasting a small relation can avoid shuffling the larger side.

Core concepts and mental model

Left semi returns only matching left rows; left anti returns nonmatching left rows. Join duplicates can multiply rows if keys are not unique.

Runnable PySpark example

```
from pyspark.sql import functions as F

trades=spark.createDataFrame([("T1","EQ"),("T2","FX")],["id","desk"])

limits=spark.createDataFrame([("EQ",100.0)],["desk","limit"])

trades.join(F.broadcast(limits),"desk","left").show()

trades.join(limits,"desk","left_anti").show()
```

Guided practical lab

Join trades with desk limits, retaining unmatched trades; inspect null limit values.

Worked solution / verification

```
assert trades.join(limits,"desk","left_anti").count()==1
```

Certification-style knowledge check

Q1: Which join returns unmatched left records? Answer: left_anti. Q2: Why broadcast? Answer: Avoid shuffling large-side data when the other side is small.

Common exam trap

Broadcasting large relations can cause memory pressure; duplicate keys may multiply results.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 06 | Spark SQL and Temporary Views

Learning focus

Spark SQL and the DataFrame API use the same underlying query engine. Temporary views are scoped to the session; global temporary views use a special global_temp namespace. SQL supports joins, CTEs, aggregations and window expressions.

Core concepts and mental model

createOrReplaceTempView registers a session-scoped name; spark.sql executes SQL. Use explicit aliases and correct null semantics.

Runnable PySpark example

```
df=spark.createDataFrame([("EQ",10.0),("EQ",-2.0),("FX",7.0)],["desk","pnl"])

df.createOrReplaceTempView("trades")

spark.sql("SELECT desk,SUM(pnl) total FROM trades GROUP BY desk").show()
```

Guided practical lab

Write SQL to count profitable trades per desk and compare with the DataFrame API.

Worked solution / verification

```
spark.sql("SELECT desk,COUNT(*) n FROM trades WHERE pnl>0 GROUP BY desk").show()
```

Certification-style knowledge check

Q1: Are SQL and DataFrames optimized by different engines? Answer: No. Q2: Is a temp view automatically a permanent table? Answer: No.

Common exam trap

A temporary view is not persistent storage; do not mistake ORDER BY for free distributed sorting.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 07 | Reading, Writing and File Formats

Learning focus

Spark reads and writes Parquet, JSON, CSV and other formats through DataFrameReader and DataFrameWriter. Parquet is columnar and schema-aware, typically preferable for analytical scans. CSV needs careful header, schema and parsing configuration.

Core concepts and mental model

Use mode("overwrite") carefully; append adds files; partitionBy creates directory-based partitioning. Reading files uses storage visible to Spark executors.

Runnable PySpark example

```
import tempfile

path=tempfile.mkdtemp()+"/trades"

df=spark.createDataFrame([("EQ",12.0),("FX",4.0)],["desk","pnl"])

df.write.mode("overwrite").parquet(path)

assert spark.read.parquet(path).count()==2
```

Guided practical lab

Write desk-partitioned Parquet and inspect directory structure; discuss high-cardinality partitionBy risks.

Worked solution / verification

```
df.write.mode("overwrite").partitionBy("desk").parquet(path+"_partitioned")
```

Certification-style knowledge check

Q1: Which format is columnar? Answer: Parquet. Q2: Does overwrite preserve existing data? Answer: No.

Common exam trap

PartitionBy a unique trade_id creates too many tiny partitions/files; CSV type inference can be unreliable.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 08 | Analytical Window Functions

Learning focus

SQL window functions calculate row-level analytics within logical partitions without collapsing rows as groupBy does. ROW_NUMBER, RANK, DENSE_RANK, LAG and running sums are common patterns.

Core concepts and mental model

Window.partitionBy(...).orderBy(...) defines row ordering. rowsBetween defines row-based frames; rangeBetween uses ordered-value ranges.

Runnable PySpark example

```
from pyspark.sql import Window,functions as F

df=spark.createDataFrame([("EQ",1,10.0),("EQ",2,15.0),("FX",1,8.0)],["desk","seq","pnl"])

w=Window.partitionBy("desk").orderBy("seq")

df.withColumn("running",F.sum("pnl").over(w.rowsBetween(Window.unboundedPreceding,Window.currentRow))).show()
```

Guided practical lab

Rank trades by P&L within each desk; compare rank() and dense_rank() on ties.

Worked solution / verification

```
w=Window.partitionBy("desk").orderBy(F.desc("pnl"))

df.withColumn("rn",F.row_number().over(w)).show()
```

Certification-style knowledge check

Q1: Does a window function collapse rows? Answer: Usually no. Q2: What distinguishes rank and dense_rank? Answer: Gaps after ties.

Common exam trap

Window ordering can be nondeterministic when order keys tie; specify a tie-breaker.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 09 | Performance Tuning and Shuffles

Learning focus

Performance depends on partition sizing, shuffle volume, join selection, file layout, skew and caching. Adaptive Query Execution (AQE) can adjust physical strategies using runtime statistics.

Core concepts and mental model

repartition() generally shuffles; coalesce() can reduce partitions without a full shuffle in common cases. cache() helps repeated reuse but consumes resources.

Runnable PySpark example

```
from pyspark.sql import functions as F

df=spark.range(100000).repartition(8)

print(df.rdd.getNumPartitions())  # traditional PySpark only

summary=df.groupBy((F.col("id")%10).alias("bucket")).count()

summary.explain("formatted")
```

Guided practical lab

Compare explain plans for repartition(8) versus coalesce(2); identify Exchange operators.

Worked solution / verification

```
df2=spark.range(10000).repartition(8).coalesce(2)

assert df2.count()==10000
```

Certification-style knowledge check

Q1: Does repartition typically shuffle? Answer: Yes. Q2: What is data skew? Answer: Uneven partition workloads.

Common exam trap

More partitions do not always improve speed; cache only reused data and unpersist when finished.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 10 | Advanced Transformations and Complex Columns

Learning focus

Spark provides array, map, struct, explode and conditional functions. Higher-order array functions enable transformations without Python UDFs. Prefer built-in expressions for optimization and remote execution.

Core concepts and mental model

explode creates multiple rows per array; explode_outer preserves null/empty inputs differently. struct groups fields; transform and filter operate on array elements.

Runnable PySpark example

```
from pyspark.sql import functions as F

df=spark.createDataFrame([("T1",[1.0,2.0,3.0])],["trade_id","sensitivities"])

df.select("trade_id",F.explode("sensitivities").alias("sensitivity")).show()

df.withColumn("doubled",F.transform("sensitivities",lambda x:x*2)).show()
```

Guided practical lab

Flatten an array of sensitivities and sum by trade ID.

Worked solution / verification

```
flat=df.select("trade_id",F.explode("sensitivities").alias("s"))

flat.groupBy("trade_id").agg(F.sum("s").alias("total")).show()
```

Certification-style knowledge check

Q1: What does explode do? Answer: Creates one row per array element. Q2: Why prefer built-ins? Answer: Catalyst visibility and lower Python overhead.

Common exam trap

explode can multiply row counts; remember null handling and nested schema shapes.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 11 | Nested Data and Higher-Order Functions

Learning focus

Nested JSON often contains structs and arrays. Spark supports field extraction, explode, transform, aggregate and filter expressions that preserve distributed execution.

Core concepts and mental model

Access struct fields with col("payload.field"); use from_json with explicit schema for JSON strings. Higher-order functions can avoid Python UDF overhead.

Runnable PySpark example

```
from pyspark.sql import functions as F

from pyspark.sql.types import StructType,StructField,StringType,ArrayType,IntegerType

s=StructType([StructField("desk",StringType()),StructField("greeks",ArrayType(IntegerType()))])

df=spark.createDataFrame([("{\"desk\":\"EQ\",\"greeks\":[2,3]}",)],["raw"])

df.select(F.from_json("raw",s).alias("p")).select("p.desk","p.greeks").show()
```

Guided practical lab

Parse JSON trade payloads and compute the sum of values in an array.

Worked solution / verification

```
parsed=df.select(F.from_json("raw",s).alias("p"))

parsed.select(F.aggregate("p.greeks",F.lit(0),lambda acc,x:acc+x).alias("sum_greeks")).show()
```

Certification-style knowledge check

Q1: Why explicit from_json schema? Answer: Stable typed parsing. Q2: Is explode a row-preserving operation? Answer: No.

Common exam trap

Do not assume JSON string fields are typed automatically; invalid JSON can produce null results.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 12 | Python UDFs, Pandas UDFs and Serialization

Learning focus

Python UDFs allow custom logic but introduce serialization and Python-worker overhead. Pandas UDFs use Arrow-based columnar transfers for supported workloads. Built-in Spark SQL functions should generally be preferred.

Core concepts and mental model

Scalar Python UDF processes values; scalar Pandas UDF processes pandas Series. UDFs can limit optimizer visibility and require dependencies on workers.

Runnable PySpark example

```
from pyspark.sql import functions as F

from pyspark.sql.types import DoubleType

@F.udf(DoubleType())

def abs_pnl(x):

    return abs(x) if x is not None else None

df=spark.createDataFrame([(-2.0,),(3.0,)],["pnl"])

df.withColumn("abs_pnl",abs_pnl("pnl")).show()
```

Guided practical lab

Rewrite the UDF with a built-in expression and compare explain plans.

Worked solution / verification

```
df.withColumn("abs_pnl",F.abs("pnl")).show()
```

Certification-style knowledge check

Q1: What is generally faster for simple arithmetic? Answer: Built-in expressions. Q2: Where do Python UDFs run? Answer: Python workers on the execution side.

Common exam trap

Do not call external services per row inside UDFs without careful scale and failure design.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 13 | Advanced Pandas UDFs and Grouped Operations

Learning focus

Pandas UDFs use Arrow-based columnar data exchange for supported Python calculations. They can improve throughput compared with ordinary row-at-a-time Python UDFs, but may still be slower than native Spark expressions.

Core concepts and mental model

Scalar Pandas UDFs operate on pandas Series. Grouped applyInPandas accepts grouped pandas DataFrames and requires an explicit return schema. Beware of groups that do not fit worker memory.

Runnable PySpark example

```
from pyspark.sql import functions as F

from pyspark.sql.types import DoubleType

import pandas as pd

@F.pandas_udf(DoubleType())

def square_values(s: pd.Series) -> pd.Series:

    return s*s

df=spark.createDataFrame([(2.0,),(3.0,)],["value"])

df.select(square_values("value").alias("square")).show()
```

Guided practical lab

Implement a Pandas UDF that converts P&L to absolute values, then compare it with F.abs().

Worked solution / verification

```
from pyspark.sql import functions as F

assert df.select(F.abs("value")).count()==2
```

Certification-style knowledge check

Q1: Which columnar format accelerates supported Pandas UDF transfers? Answer: Arrow. Q2: Are native Spark expressions usually preferable for simple math? Answer: Yes.

Common exam trap

Pandas UDFs require compatible pandas and Arrow dependencies; grouped operations can have severe skew-related memory pressure.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 14 | Intermediate Review and Assessment

Learning focus

This checkpoint consolidates DataFrames, SQL, joins, windows, partitioning, storage and optimization. Use a realistic multi-step trade-reporting task to test integration rather than memorizing isolated APIs.

Core concepts and mental model

Core review: action vs transformation, join types, null behavior, window frames, Parquet, explain, AQE, UDF trade-offs.

Runnable PySpark example

```
from pyspark.sql import functions as F

trades=spark.createDataFrame([("T1","EQ",10.0),("T2","EQ",-4.0),("T3","FX",8.0)],["id","desk","pnl"])

report=trades.groupBy("desk").agg(F.sum("pnl").alias("net_pnl"),F.count("*").alias("n"))

report.show()
```

Guided practical lab

Capstone: ingest trade rows, validate, join limits, compute desk P&L, rank desks, write Parquet.

Worked solution / verification

```
assert report.filter("desk = 'EQ'").first()["net_pnl"]==6.0
```

Certification-style knowledge check

Q1: Which join returns only unmatched left keys? Answer: left_anti. Q2: Which action displays a plan? Answer: explain(). Q3: Which operation triggers execution? Answer: count().

Common exam trap

No verified prior assessment scores are available; treat this chapter as review, not a completed exam result.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 15 | Structured Streaming Fundamentals

Learning focus

Structured Streaming models an unbounded stream as a continuously updated table. readStream creates streaming DataFrames; writeStream starts a query. Most DataFrame transformations remain familiar, but actions and output modes differ.

Core concepts and mental model

Sources include files, Kafka and rate. Triggers schedule processing. Checkpoints track progress. Streaming queries use start(), awaitTermination(), stop().

Runnable PySpark example

```
from pyspark.sql import functions as F

stream=spark.readStream.format("rate").option("rowsPerSecond",2).load()

query=stream.select("timestamp","value").writeStream.format("console").outputMode("append").trigger(processingTime="5

seconds").start()

# Stop after observing output: query.stop()
```

Guided practical lab

Build a file-stream pipeline from JSON files with explicit schema and a persistent checkpoint.

Worked solution / verification

```
schema="trade_id STRING, desk STRING, pnl DOUBLE"

# stream=spark.readStream.schema(schema).json(input_dir)

# stream.writeStream.format("parquet").option("path",output_dir).option("checkpointLocation",ckpt).start()
```

Certification-style knowledge check

Q1: What starts streaming execution? Answer: writeStream.start(). Q2: Can you call collect() directly on a streaming DataFrame? Answer: Not as a regular batch action.

Common exam trap

A streaming DataFrame is not a static snapshot; schema inference for streaming files is typically disabled by default.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 16 | Streaming Sources, Sinks, Triggers and Output Modes

Learning focus

File and Kafka sources feed micro-batches; console, memory, file and Kafka sinks have different capabilities. Append, Update and Complete determine which result rows are emitted.

Core concepts and mental model

availableNow processes available input then terminates; processingTime schedules periodic triggers. foreachBatch exposes each micro-batch as a batch DataFrame.

Runnable PySpark example

```
stream=spark.readStream.format("rate").option("rowsPerSecond",3).load()

q=stream.writeStream.format("console").outputMode("append").trigger(availableNow=True).start()

q.awaitTermination()
```

Guided practical lab

Identify valid combinations of output modes and sinks for a stateless stream versus an aggregation.

Worked solution / verification

```
from pyspark.sql import functions as F

# For streaming aggregations, mode support depends on query and sink.

# windowed=stream.groupBy(F.window("timestamp","5 minutes")).count()
```

Certification-style knowledge check

Q1: Does Append emit all aggregate updates? Answer: No. Q2: What is availableNow? Answer: A trigger that processes currently available input and stops.

Common exam trap

Kafka value is binary; decode before JSON parsing. Not every sink supports every output mode.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 17 | Event-Time Windows, Watermarks and Stateful Processing

Learning focus

Event time is when an event occurred; processing time is when Spark processes it. Tumbling windows do not overlap; sliding windows can overlap. Stateful aggregation stores partial results across batches.

Core concepts and mental model

withWatermark(event_time, delay) tracks event-time progress to bound eligible state. Append-mode window output is finalized after watermark passes window end; progress is batch-dependent.

Runnable PySpark example

```
from pyspark.sql import functions as F

stream=spark.readStream.format("rate").option("rowsPerSecond",2).load()

agg=(stream.withWatermark("timestamp","10 minutes")

    .groupBy(F.window("timestamp","5 minutes")).count())

# Choose a compatible append sink and checkpoint before start().
```

Guided practical lab

Build a 5-minute window with a 10-minute watermark; explain what happens to a very late record.

Worked solution / verification

```
agg.explain()

# A late event may be dropped if its window has already been finalized.
```

Certification-style knowledge check

Q1: Is a 10-minute watermark a fixed wall-clock wait? Answer: No. Q2: Does a sliding window overlap? Answer: It can.

Common exam trap

A watermark does not indiscriminately filter every old row; deduplication keys determine business semantics.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 18 | Streaming Checkpoints, Recovery and foreachBatch

Learning focus

Checkpoints persist source progress, committed-batch information and state when applicable. Restarting with a compatible checkpoint can resume processing. Exactly-once effects depend on source, sink and idempotent/transactional behavior.

Core concepts and mental model

foreachBatch(batch_df,batch_id) is at-least-once by default. Use durable batch identifiers and coordinated writes to make retries safe. Stateful checkpoint changes may be incompatible.

Runnable PySpark example

```
def process_batch(batch_df,batch_id):
    # In production: write transactionally and dedupe by query+batch id.
    print("batch",batch_id,"rows",batch_df.count())
# query=stream.writeStream.foreachBatch(process_batch).option("checkpointLocation",ckpt).start()
```

Guided practical lab

Run a file-to-Parquet availableNow query twice with the same checkpoint and new input files; verify the output count.

Worked solution / verification

```
assert 3+2==5  # expected after first batch (3) and second batch (2)
# Real validation: spark.read.parquet(output_dir).count() == 5
```

Certification-style knowledge check

Q1: Does cache replace streaming checkpointing? Answer: No. Q2: Is foreachBatch automatically exactly-once? Answer: No.

Common exam trap

Do not delete production checkpoints casually; state schema and query changes can break recovery.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 19 | Spark Connect Architecture

Learning focus

Spark Connect separates a lightweight client from the remote Spark driver. Clients serialize unresolved logical plans using Protocol Buffers and send them over gRPC; the server analyzes and executes the plan.

Core concepts and mental model

Spark Connect does not replace the Spark engine. RDDs, SparkContext and direct JVM internals are unavailable through its API. Arrow can transport tabular results.

Runnable PySpark example

```
from pyspark.sql import SparkSession

# Requires a running compatible Spark Connect server.

spark=SparkSession.builder.remote("sc://localhost:15002").getOrCreate()

df=spark.range(100)

print(df.filter("id > 50").count())
```

Guided practical lab

Explain where logical plans, physical optimization, executor tasks and result transport occur.

Worked solution / verification

```
assert spark.range(100).filter("id > 50").count()==49
```

Certification-style knowledge check

Q1: Which protocol? Answer: gRPC. Q2: Are RDDs supported? Answer: No. Q3: Does Catalyst still run? Answer: Yes, server-side.

Common exam trap

Do not access spark.sparkContext or df.rdd from Spark Connect; collect() can still overload the client.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

DAY 20 | Spark Connect Applications, Testing and Remote Data

Learning focus

Build maintainable remote PySpark applications using pure DataFrame transformations, a connection layer and small integration tests. Client-side paths are not automatically accessible to the server. UDF dependencies must exist in remote workers.

Core concepts and mental model

Use SparkSession.builder.remote(endpoint), environment variables for endpoints, and pytest for transformation tests. DataFrame API operations remain lazy; actions trigger server execution.

Runnable PySpark example

```
from pyspark.sql import functions as F

def validate(df):

    return df.filter(F.col("trade_id").isNotNull() & F.col("pnl").isNotNull())

def desk_report(df):

    return df.groupBy("desk").agg(F.sum("pnl").alias("total_pnl"),F.count("*").alias("n"))

trades=spark.createDataFrame([("T1","EQ",100.0),("T2","EQ",-20.0),("T3","FX",50.0)],["trade_id","desk","pnl"])

desk_report(validate(trades)).show()
```

Guided practical lab

Implement validate_trades, enrich_trades and aggregate_by_desk functions; add pytest assertions and explain client/server boundaries.

Worked solution / verification

```
out=desk_report(validate(trades))

assert out.filter("desk = 'EQ'").first()["total_pnl"]==80.0
```

Certification-style knowledge check

Q1: Where must remote read paths be accessible? Answer: Spark server/executors. Q2: Are Python UDF dependencies only local? Answer: No.

Common exam trap

Spark Connect does not upload local files automatically; tests require a running compatible server.

Reflection and practice

Explain the result in your own words. Modify one input value and predict the output before rerunning. Record whether you completed the lab and your quiz score; no prior verified scores are assumed.

CROSS-CHAPTER PRACTICE PROJECT

Build a desk-level risk and P&L reporting pipeline from a small trade dataset. Define an explicit schema, validate nulls, join desk reference data, aggregate net P&L, rank desks using window functions, write Parquet, and inspect the physical plan. Then implement an incremental file-stream ingestion with checkpointing and discuss how Spark Connect changes the client boundary.

Suggested acceptance criteria

1. Input schema is explicit. 2. Missing values are handled intentionally. 3. Joins do not unexpectedly multiply trades. 4. Desk P&L is validated against a hand-calculated total. 5. Output is written in a columnar format. 6. An explain plan is reviewed. 7. Streaming checkpoint and sink paths are distinct. 8. A Spark Connect client can run the same DataFrame transformation logic when a compatible server is available.

Final review questions

Why does groupBy usually shuffle? What makes a left anti join different from a left semi join? How do event-time windows differ from processing-time triggers? Why is foreachBatch at-least-once by default? Which APIs are missing in Spark Connect? Where must server-side UDF dependencies be installed?

Next chapters

Day 21: deployment and execution modes. Days 22-24: Pandas API on Spark. Days 25-28: integration labs, exam traps and mock examinations.

Official documentation

https://spark.apache.org/docs/latest/api/python/

https://spark.apache.org/docs/latest/streaming/index.html

https://spark.apache.org/docs/latest/spark-connect-overview.html

Day 20 / 28 — Spark Connect: Building Remote PySpark Applications

Instructor handbook and learner workbook | Theory, architecture, runnable examples, five labs, solutions, exam practice and troubleshooting.

Prerequisite: an accessible, compatible Spark Connect server. Local PySpark client installation alone does not launch the server.

1. Objectives & agenda

• Understand Spark Connect remote execution and client/server boundaries

• Build modular PySpark transformations and an application entry point

• Handle remote paths, Python UDF dependencies and version compatibility

• Test DataFrame logic and diagnose common failures

• Practice 5 labs and 12 certification-style questions

2. Architecture refresher

• Client: VS Code / Python / Spark Connect DataFrame API

• Transport: gRPC requests containing Protocol Buffer logical plans

• Server: Spark driver analyzes plans with Catalyst and coordinates execution

• Executors: perform distributed tasks; Arrow can transfer result batches

• Spark Connect changes the interface, not the underlying Spark engine

3. Traditional PySpark vs Connect

• Traditional PySpark typically uses Py4J to communicate with the driver JVM

• Connect uses gRPC and a decoupled remote client

• DataFrames and Spark SQL are supported in both

• SparkContext, RDD APIs and private JVM objects are not supported in Connect

• Neither interface guarantees faster joins, shuffles or aggregations

4. Environment setup

• Create a virtual environment and install compatible Spark Connect client and pytest

• Example Spark 4.0 client: pip install "pyspark-client==4.0.1" pytest

• Start a compatible Spark Connect server separately

• Configure SPARK_REMOTE=sc://localhost:15002 or use builder.remote(...)

• Check network access, version compatibility and server permissions

```
python -m venv .venv
# activate environment, then:
pip install "pyspark-client==4.0.1" pytest
```

5. Project structure

• spark-connect-risk/

• app.py — application entry point and session lifecycle

• transformations.py — pure DataFrame transformations

• test_transformations.py — automated integration tests

• requirements.txt — pinned compatible dependencies

Remote session

```
import os
from pyspark.sql import SparkSession
spark = (SparkSession.builder
         .remote(os.getenv("SPARK_REMOTE", "sc://localhost:15002"))
         .getOrCreate())
```

transformations.py

```
from pyspark.sql import functions as F

def validate_trades(df):
    return df.filter(F.col("trade_id").isNotNull() &
                     F.col("desk").isNotNull() &
                     F.col("pnl").isNotNull())

def enrich_trades(df):
    return df.withColumn("pnl_category",
        F.when(F.col("pnl") > 0, "PROFIT")
         .when(F.col("pnl") < 0, "LOSS").otherwise("FLAT"))

def aggregate_by_desk(df):
    return df.groupBy("desk").agg(
        F.sum("pnl").alias("total_pnl"),
        F.count("*").alias("trade_count"),
        F.avg("pnl").alias("average_pnl"))
```

app.py

```
from transformations import validate_trades, enrich_trades, aggregate_by_desk
from pyspark.sql import SparkSession
import os

def main():
    spark = SparkSession.builder.remote(
        os.getenv("SPARK_REMOTE", "sc://localhost:15002")).getOrCreate()
    trades = spark.createDataFrame([
        ("T101", "Equity", 100.0), ("T102", "Rates", 50.0),
        ("T103", "Equity", -20.0), ("T104", "FX", 80.0),
        ("T105", "Equity", 40.0), ("T106", "Rates", None)],
        ["trade_id", "desk", "pnl"])
    report = aggregate_by_desk(enrich_trades(validate_trades(trades)))
    report.orderBy("desk").show()
    spark.stop()

if __name__ == "__main__":
    main()
```

6. Validation & enrichment

• Validation filters null trade_id, desk and pnl

• Enrichment assigns PROFIT, LOSS or FLAT from pnl sign

- Return a new DataFrame; avoid actions inside reusable transformations
- Keep business rules separate from connection and presentation logic
- For real systems, route invalid trades to a quarantine table instead of silently discarding

7. Desk aggregation

- Group by desk; compute sum(pnl), count(*), avg(pnl)
- Sample Equity: 100 - 20 + 40 = 120 across 3 trades
- Rates: 50; FX: 80; one Rates null-pnl record excluded by validation
- Aggregation is lazy until an action such as show()
- Consider decimal types for precise financial calculations in production

8. Remote storage

- spark.read.parquet("/data/trades") resolves in remote execution environment
- A Windows client path is not automatically visible to a Linux server
- Use storage accessible to Spark: cloud object storage or governed volumes
- Databricks example: /Volumes/catalog/schema/volume/path
- Verify catalog permissions, object access and executor visibility

9. Client vs server dependencies

- Client builds Spark expressions and submits logical plans
- Built-in Column operations execute remotely within Spark
- Python UDF code executes in remote Python worker environment
- Server-side workers need compatible Python packages and dependencies
- Prefer built-in expressions when possible for optimizer visibility

10. Testing strategy

- Test small DataFrames with deterministic expected outputs
- Use pytest fixtures to create and stop a remote SparkSession
- Check validation, enrichment and aggregate totals separately
- Collect only small bounded test results
- Tests requiring Spark Connect server are integration tests

11. Lab 1 — risk flags

- Add risk_flag: pnl < -50 => HIGH_LOSS
- -50 <= pnl < 0 => LOSS; pnl >= 0 => NORMAL
- Use when().when().otherwise()
- Test -100, -20 and +50 with exact expected categories

• Decide explicitly how null pnl should be handled

Lab solutions

```
from pyspark.sql import functions as F

def add_risk_flag(df):
    return df.withColumn("risk_flag",
        F.when(F.col("pnl").isNull(), "MISSING")
         .when(F.col("pnl") < -50, "HIGH_LOSS")
         .when(F.col("pnl") < 0, "LOSS")
         .otherwise("NORMAL"))

def build_desk_report(df):
    return df.groupBy("desk").agg(
        F.sum("pnl").alias("total_pnl"),
        F.count("*").alias("trade_count")).orderBy("desk")
```

Tests

```
def test_aggregation(spark):
    df = spark.createDataFrame([
        ("T101", "Equity", 100.0),
        ("T102", "Equity", -20.0),
        ("T103", "Rates", 50.0)],
        ["trade_id", "desk", "pnl"])
    totals = {r["desk"]: r["total_pnl"]
              for r in aggregate_by_desk(df).collect()}
    assert totals == {"Equity": 80.0, "Rates": 50.0}
# Run: pytest -v
```

12. Lab 2 — reusable reporting

• Implement build_desk_report(df) returning DataFrame

• Group by desk; compute total_pnl and trade_count

• Sort by desk for deterministic inspection

• Validate with a small test dataset

• Do not create a new SparkSession inside each transformation

13. Lab 3 — SQL equivalence

• Create a temporary view using createOrReplaceTempView

• Write GROUP BY desk, SUM(pnl) in spark.sql

• Compare DataFrame API and SQL results after deterministic sorting

• Both use Spark SQL engine and Catalyst optimization

• Avoid assuming collect() order without ORDER BY

14. Lab 4 — remote path diagnosis

• Client Windows path: C:/Users/me/Desktop/trades.csv

• Remote Spark server on Linux cannot automatically see it

• Upload to a shared accessible location or governed volume

• Confirm file format, permissions and path spelling

• Avoid copying sensitive trade files into ungoverned storage

15. Lab 5 — execution boundaries

• Column expression construction: client

• Logical plan analysis and optimization: server

• Distributed join and aggregation: executors

• collect() result transfer: server to client

• Python UDF execution: remote Python workers

16. Troubleshooting

• Connection refused: server unavailable, wrong host or port

• Unsupported sparkContext or df.rdd: use DataFrame APIs

• File not found: check remote path and permissions

• ModuleNotFoundError in UDF: install package on remote workers

• Client memory error: avoid unbounded collect()/toPandas()

Troubleshooting workflow: (1) verify server reachability; (2) verify client/server versions; (3) inspect Spark-side logs; (4) isolate transformation using a small DataFrame; (5) check storage permissions and Python worker dependencies.

17. Exam traps

• Connect does not eliminate the Spark driver

• Remote Spark does not automatically read client-local files

• Spark Connect does not support RDD or SparkContext access

• Client-only UDF dependencies are insufficient

• A Spark Connect application can still have skew and expensive shuffles

18. Knowledge check — questions 1–6

• 1. Main purpose of Connect? A Replace Spark B Decouple client C Remove executors

• 2. Protocol? A FTP B gRPC C SSH

• 3. Query optimizer location? A Client B Browser C Server

• 4. Unsupported API? A filter B groupBy C SQL D RDD

• 5. builder.remote() sets? A Endpoint B Partition count C Checkpoint

• 6. Modular transformations improve? A Testability B Disable Catalyst C Caching

19. Knowledge check — questions 7–12

• 7. File path must be visible to? A Client only B Remote Spark

• 8. UDF dependencies must exist? A Client only B Remote workers

• 9. Action returning all rows? A filter B select C collect

• 10. Connection refused likely? A Endpoint/server B Null data

• 11. Simple arithmetic: A Python UDF B Built-in expression

• 12. Tests can use? A Small DataFrames B Kafka only

20. Quiz answers & reasoning

• 1 B — decoupled client-server interface; 2 B — gRPC

• 3 C — server-side Catalyst; 4 D — RDD APIs unsupported

• 5 A — remote endpoint; 6 A — modular code easier to test

• 7 B — server-accessible path; 8 B — remote Python workers

• 9 C — collect transfers rows; 10 A — connection/endpoint issue

• 11 B — optimizer-friendly built-ins; 12 A — small test DataFrames

21. Recap & next lesson

• 20 of 28 lessons delivered (~71%); no verified quiz scores supplied

• Can you connect, transform, test, diagnose paths and dependencies?

• Suggested practice target: 11/12 quiz answers and all five labs

• Day 21: local, client and cluster deployment modes; spark-submit

• References: spark.apache.org/docs/latest/spark-connect-overview.html

Trainer delivery notes

• Use 20 minutes for architecture discussion and 30 minutes for setup; learners need a working server before running code.

• For the labs, let learners attempt each task before revealing the solution. Encourage assert-based validation.

• Use the quiz questions before showing the answer slide. Do not present the training score as an official certification passing score.

• Financial examples are simplified for training; production P&L and risk-limit logic require validated business definitions, decimal precision and data quality controls.

Official references

https://spark.apache.org/docs/latest/spark-connect-overview.html

https://spark.apache.org/docs/latest/api/python/

https://spark.apache.org/docs/latest/submitting-applications.html